

Resumen teórico — Sistemas Operativos

Notas del curso: *Memoria virtual*

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico del capítulo *Memoria virtual* de las *Notas del curso* de Sistemas Operativos (Marcelo Arroyo, UNRC). Se desarrollan las técnicas que extienden la memoria física disponible usando disco: *swapping* o *demand paging*, mapeo de archivos en memoria (`mmap`), carga de programas bajo demanda, crecimiento dinámico del stack, memoria compartida entre procesos, *copy on write* en `fork()` y los algoritmos clásicos de reemplazo de páginas (FIFO, second chance / clock, LRU / NFU, NRU, working set). Cierra con el caso de estudio del kernel de GNU/Linux. El objetivo es servir de material de estudio para preparar los prácticos asociados a memoria virtual y el segundo parcial.

Índice

1. Introducción: memoria virtual y swapping	2
1.1. Swap-in y swap-out	2
1.2. Estado de cada página	2
1.3. Extender la RAM con disco	3
1.4. Tratamiento del page fault	3
1.5. Swap area	3
2. Mapeo de archivos en memoria	3
2.1. Cómo funciona	4
2.2. Ejemplo de uso	4
2.3. Ilustración del cambio en la tabla de páginas	4
2.4. Manejo del page fault sobre un mapping	5
3. Carga de programas bajo demanda	5
4. Stack con crecimiento dinámico	6
5. Memoria compartida	6
5.1. Llamadas al sistema (UNIX)	6
5.2. Bibliotecas compartidas	7
5.3. Liberación y contador de referencias	7
6. Copiar al escribir (Copy on Write, COW)	7
6.1. La optimización	7
6.2. Qué pasa cuando alguien quiere escribir	7
7. Algoritmos de reemplazo de páginas	8
7.1. FIFO	8
7.2. Second chance (algoritmo del reloj, <i>clock</i>)	8
7.3. Menos recientemente usada (LRU)	9
7.4. No recientemente usada (NRU)	9

7.5. Working set	10
7.6. Otras consideraciones	10
8. Caso de estudio: GNU/Linux	11
9. Cierre	11

1. Introducción: memoria virtual y swapping

La evolución del hardware permite que las CPUs modernas soporten *enormes* espacios de direcciones: una CPU moderna puede manejar direcciones de 64 bits (128 terabytes) o más. Esto habilita el desarrollo de programas grandes.

Importante

En un sistema moderno, la suma de memoria RAM requerida por los procesos suele ser **mayor** a la disponible físicamente en el hardware. La **memoria virtual** es la técnica que permite ejecutar esos programas, manteniendo en RAM sólo las partes en uso en cada momento.

Idea central. Un proceso no necesita estar cargado en memoria *completamente*: sólo necesita las partes que está usando ahora. Con el esquema de paginado visto en el capítulo anterior, es posible cargar (*swap-in*) o desalojar (*swap-out*) *partes* de un proceso con la granularidad del tamaño de página.

1.1. Swap-in y swap-out

- Un **swap-in** se produce cuando el proceso referencia una página que reside en disco: hay que traerla a RAM.
- Un **swap-out** se produce cuando el sistema necesita liberar una página de RAM (por ejemplo, porque no quedan páginas libres). Comúnmente lo invoca el *memory allocator*.

Este proceso de cargar/desalojar páginas en disco se conoce como **swapping** o **demand paging**.

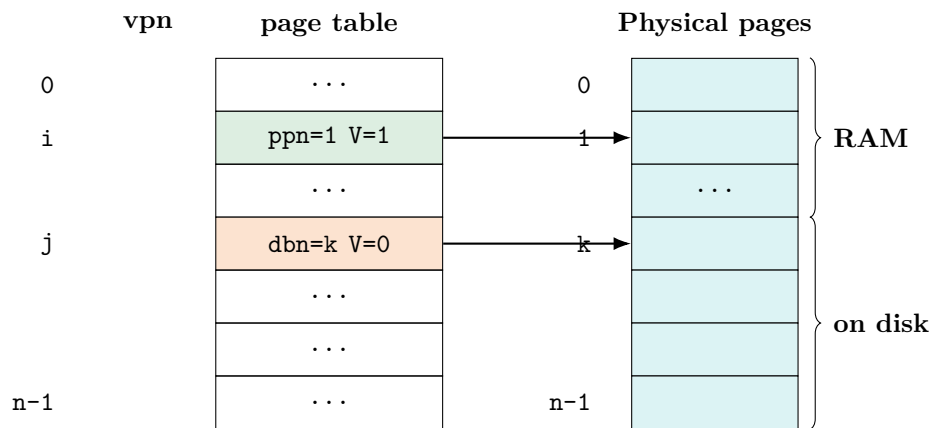
1.2. Estado de cada página

Para implementar memoria virtual, por cada página se debe mantener su *estado*: si está mapeada a una página física en RAM, o si está guardada en un bloque de disco. Esto se hace almacenando en la *pte*:

- el **physical page number** (ppn) si la página está en RAM, con el bit **V** (*valid* o *present* en algunas arquitecturas) en 1, o bien
- el **disk block number** (dbn) si la página reside en disco, con el bit V en 0.

Con esta convención, el hardware genera automáticamente un *page fault* cuando se referencia una página con $V = 0$ y le da al kernel la oportunidad de hacer el swap-in correspondiente.

1.3. Extender la RAM con disco



Idea intuitiva

Cada *pte* le dice al hardware “dónde” está realmente la página: si en RAM (vía **ppn**) o si en disco (vía **dbn**). La RAM se comporta entonces como una caché del conjunto total de páginas “virtuales”. El espacio de direcciones del proceso puede ser mayor que la RAM física.

1.4. Tratamiento del page fault

Cuando una instrucción accede a la página j con $V = 0$, la CPU genera una excepción. El kernel ejecuta un **swap-in**:

1. Reservar una página física en RAM (sea $ppn = X$).
2. Leer el contenido del bloque k de disco en la página X .
3. Mapear la *pte* j : $ppn = X$ y $V = 1$.

Al retornar de la excepción, la CPU re-ejecuta la instrucción que falló y esta vez sí puede acceder al contenido.

1.5. Swap area

El **swap area** es la región de disco donde se almacenan las páginas que están fuera de RAM. Puede ser un **archivo** dentro de un sistema de archivos común o una **partición** con un formato específico de swap. Por ejemplo, GNU/Linux permite ambas configuraciones; una partición con formato de tipo *swap* mejora sustancialmente el rendimiento respecto a un archivo, porque evita la indirección del sistema de archivos.

2. Mapeo de archivos en memoria

Una aplicación natural y muy potente de la memoria virtual es el **mapeo de archivos en memoria** (*memory-mapped files*). La llamada al sistema típica es:

```
va = mmap(fd, mode);
```

Definición

`mmap` crea un *mapping* de direcciones lógicas a los contenidos del archivo en disco y retorna la dirección lógica base del contenido mapeado en memoria. Tras esa llamada, el proceso puede acceder al archivo como si fuera un arreglo de bytes en memoria, sin usar `read` o `write`.

2.1. Cómo funciona

El contenido del archivo se divide lógicamente en bloques del tamaño de una página. `mmap` no asigna memoria física ni carga el archivo al invocarse: sólo prepara el espacio de direcciones (setea *ptes* indicando que los datos están en los bloques de disco correspondientes, con bit $V = 0$).

La carga real ocurre **bajo demanda**, de forma transparente al proceso: cuando el programa referencia `va[i]`, el hardware genera un page fault, el kernel asigna una página, carga en ella el bloque correspondiente al offset i y deja el *pte* válido. Las siguientes lecturas/escrituras a esa misma página ya van directo a RAM.

En general, la dirección de comienzo del mapping en sistemas tipo UNIX es el puntero `brk` del proceso, que marca el inicio del bloque de espacio de direcciones libre. `mmap` avanza este puntero.

Una llamada al sistema `unmmap(va)` libera el mapping y la memoria física asociada.

2.2. Ejemplo de uso

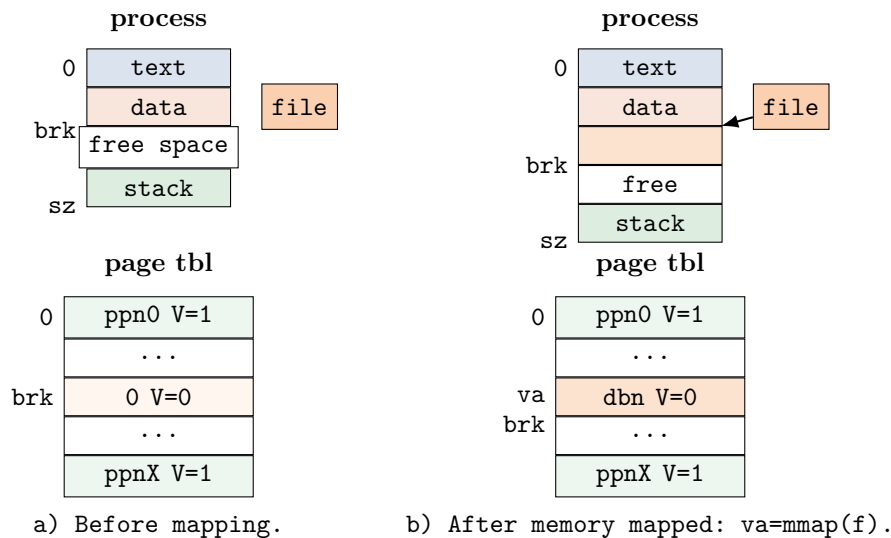
```
// myfile.dat contiene mas de 4096 bytes. Despues de mmap(),
// se ve como un arreglo de chars buf[]:
//
// buf --> +-----+ 0
// | Hello world\0 |
// | ... | 4095
// +-----+ 4096
// | Second page\0 | ...
// | padding (zeroes) by mmap |
// +-----+ 8192

int fd = open("myfile.dat", O_RDONLY);
char *buf = mmap(fd);

printf("%s\n", buf); // output: Hello world
printf("%s\n", buf + 4096); // output: Second page
printf("%s\n", buf + 7005); // output: cadena vacia (ceros)
printf("%s\n", buf + 8192); // page fault (quizá)
```

2.3. Ilustración del cambio en la tabla de páginas

Antes del `mmap`, en la tabla de páginas figura el área del proceso (text, data, stack) y, a partir de `brk`, entradas inválidas. Después del `mmap`, las entradas que cubren la región mapeada tienen $V = 0$ con `dbn` apuntando a bloques del archivo.



2.4. Manejo del page fault sobre un mapping

Cuando el programa accede a la región mapeada y la *pte* tiene $V = 0$, el hardware genera la excepción. El kernel verifica que la dirección lógica sea válida (por ejemplo, que el campo *ppn* no sea cero, indicando que la entrada está *configurada* para mapping). Si así es, procesa la excepción:

1. Reservar (*alloc*) una página física, sea $ppn = \alpha$.
2. Cargar el bloque *dbn* de disco en la página obtenida.
3. Mapear: setear $pte.ppn = \alpha$ y los flags ($V = 1$) en la *pte* correspondiente.

Al retornar de la excepción, la CPU vuelve a ejecutar la instrucción que la causó y accede exitosamente al contenido.

Escrituras y bit dirty. Si el proceso modifica el contenido en memoria del archivo, al liberar el mapping el SO puede *escribir esos datos de vuelta al archivo*. El bit **dirty** (D) de la *pte* indica qué páginas fueron modificadas, de modo que sólo se escriben las que efectivamente cambiaron.

3. Carga de programas bajo demanda

La llamada al sistema `exec(program-file, args)` debe cargar el contenido del ejecutable a memoria. Si el programa es grande, hacerlo de manera completa requiere tiempo y memoria, gran parte de los cuales podrían no usarse nunca.

Idea intuitiva

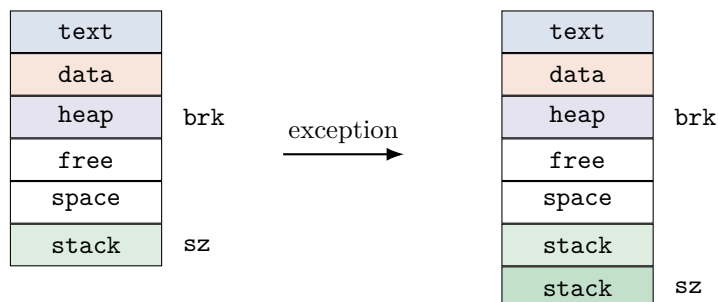
Si `exec` *mapea* los segmentos cargables del ejecutable con `mmap` y eventualmente carga sólo la página de código que contiene el *entry point*, es posible realizar la carga del programa **bajo demanda**: las demás páginas se traen a RAM a medida que la ejecución avanza y referencia nuevas instrucciones o datos.

Esta optimización es esencial en SO modernos: arrancar un binario grande es virtualmente instantáneo porque, salvo una página, todo el ejecutable está sólo *mapeado*, no cargado.

4. Stack con crecimiento dinámico

Otro problema de la gestión de memoria es decidir cuánta memoria reservar para el *stack* de cada proceso. Si reservamos poco, un programa con recursión profunda se queda sin pila; si reservamos mucho, desperdiciamos memoria.

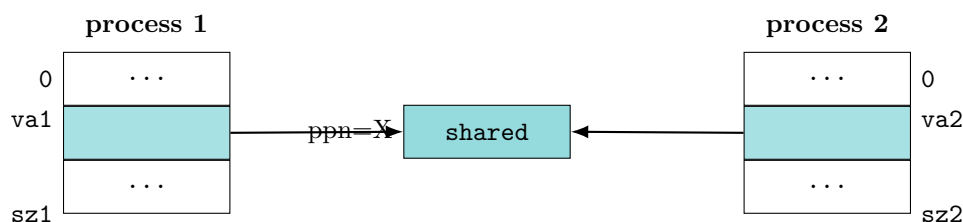
Con memoria virtual la solución es elegante: se mapea inicialmente **una sola página** en una dirección lógica alta y el SO asigna y mapea *más páginas bajo demanda*, a medida que el proceso intenta escribir sobre el tope del stack y eso genera una excepción.



De esta manera el stack *crece bajo demanda*. El heap, simétricamente, suele expandirse hacia arriba (en términos lógicos) asignándole más espacio mediante la llamada al sistema `sbrk(size)`.

5. Memoria compartida

Para que dos o más procesos (o el kernel y los procesos) se comuniquen usando memoria, se requiere que *uno o más bloques de memoria físicos pertenezcan a sus espacios de direcciones simultáneamente*. Esto se logra mapeando en cada proceso un espacio de direcciones lógicas que resuelve a las *mismas direcciones físicas*.



Las direcciones lógicas va_1 y va_2 comúnmente son *distintas* en cada proceso, pero ambas están mapeadas al mismo número de página física $ppn = X$.

5.1. Llamadas al sistema (UNIX)

Los SO tipo UNIX proveen un conjunto de llamadas para compartir memoria:

- `int shmget(key, size, flags)`: crea u obtiene un descriptor para un bloque de memoria compartida con identificador `key` de tamaño `size`.
- `void *shmat(m)`: obtiene la dirección lógica del descriptor `m` retornado por `shmget`.
- `int shmdt(address)`: libera (*detach*) el mapping.

Importante

Los procesos que se comuniquen mediante memoria compartida *deben* usar algún mecanismo de sincronización (semáforos, mutexes, ...) para evitar condiciones de carrera en el acceso a los datos compartidos. La memoria compartida *no* es un canal sincronizado por sí mismo.

5.2. Bibliotecas compartidas

Esta técnica, junto con el mapeo de archivos en memoria, es la que se usa para mapear una **biblioteca compartida** (*shared library*) en el espacio de direcciones de un proceso. Así, varios procesos pueden “ver” las mismas páginas físicas de `libc`, `libssl`, etc., en lugar de cargar copias separadas.

5.3. Liberación y contador de referencias

La memoria compartida complica la liberación de memoria cuando un proceso termina (por ejemplo, en un `exit()`): hay que liberar sólo las páginas que *no* son compartidas con otro proceso.

Una solución estándar es asociar a cada *shared memory region* un **contador de sharing** (o referencias):

- Se incrementa con cada `va = shmat(m)`.
- Se decrementa con cada `shmdt(va)`.
- Cuando llega a cero, es seguro liberar la memoria compartida.

6. Copiar al escribir (Copy on Write, COW)

La llamada al sistema `fork()` crea un proceso hijo *copía* del padre. Una implementación naïve copiaría completamente la imagen del proceso padre en el hijo, lo cual puede requerir gran cantidad de memoria y mucho tiempo, especialmente si luego el hijo va a hacer `exec` y reemplazar toda esa memoria.

6.1. La optimización

En lugar de copiar todo, se copia sólo la **tabla de páginas**, haciendo que ambos procesos (padre e hijo) compartan inicialmente la memoria física. Para mantener el confinamiento de cada proceso, en *ambas* tablas de páginas se quitan los permisos de **escritura** ($W = 0$) en cada *pte*, dejando la memoria compartida como **sólo de lectura**.

A cada proceso debería asociársele una *shared memory block* que describa toda su zona de direcciones COW.

6.2. Qué pasa cuando alguien quiere escribir

Cuando uno de los procesos intenta escribir en una página compartida (*ppn*) ocurre lo siguiente:

1. La CPU genera una excepción porque no tiene permiso de escritura.
2. El kernel captura la excepción y reconoce que se debe a la falta de permisos, pero la dirección es *válida y compartida*.
3. Asigna (*alloc*) una nueva página, sea con $ppn = n$.
4. Copia el contenido de la página original en la nueva.
5. Mapea la nueva página en el proceso que intentó escribir: $pte.ppn = n$ y $pte.W = 1$.
6. En el otro proceso (padre o hijo, según corresponda) también restaura $W = 1$ en su *pte*, porque ahora ese proceso es el único dueño de la página original.

Como en toda excepción recuperable, el kernel prepara la CPU para que **re-intente** la ejecución de la instrucción que provocó la falta.

Definición

Este mecanismo se conoce como **copy on write (COW)**: la copia real se difiere hasta el momento en que efectivamente se necesita porque alguien escribe. Mientras los procesos sólo lean, no hay duplicación de memoria.

Para generalizar COW se necesita llevar la pista de las páginas compartidas entre procesos, comúnmente usando *shared memory regions* en los descriptores de tareas (los *task structs* del kernel).

Idea intuitiva

COW es lo que hace que `fork()` + `exec()` sea barato: el padre no se “clona” físicamente; sólo se prepara para clonarse si hace falta. Como `exec()` suele descartar toda la memoria del hijo, la copia nunca llega a producirse.

7. Algoritmos de reemplazo de páginas

Cuando el sistema requiere asignar memoria (por ejemplo `allocpage()`), ya sea para un proceso o para el kernel, puede ocurrir que no exista ninguna página libre. Esta situación se conoce como **page fault**.

Con swapping el sistema puede elegir una página (la **víctima**) que actualmente está en uso, hacerle un *swap-out* a disco y reutilizar el frame para satisfacer el requerimiento.

Importante

El problema central es: **¿cómo elegir la víctima?** Lo ideal sería que fuera una página que *no* va a referenciarse en mucho tiempo (o nunca más). Pero esto no es computable: requiere predecir el futuro. Los algoritmos prácticos intentan aproximarlos con heurísticas basadas en el comportamiento pasado.

7.1. FIFO

El sistema mantiene una cola de las páginas en uso: la última asignada queda al final. Ante un page fault, el algoritmo **FIFO** elige como víctima la página a la *cabeza* de la cola (la que entró hace más tiempo). La nueva página se encola al final.

Crítica. FIFO es muy simple, pero puede elegir como víctima una página que se usa con **altísima frecuencia** (por ejemplo, la página de código con el bucle principal del programa), sólo por haber entrado primero.

7.2. Second chance (algoritmo del reloj, *clock*)

Es una corrección sencilla de FIFO que aprovecha el bit **A** (*accessed*) que el hardware setea cuando una página es leída o escrita.

En vez de tomar la página de la cabeza directamente, se inspecciona primero su bit *A*:

- Si $A = 1$, se hace $A = 0$ y la página se mueve al final de la cola, dándole una “segunda oportunidad”.
- Si $A = 0$, se la elige como víctima.

Esto continúa hasta encontrar una página a la cabeza con $A = 0$. La idea es que la víctima elegida *no* fue accedida desde el último recorrido.

Variante: el algoritmo del reloj. En lugar de usar una cola, se usa una **lista circular** de las páginas usadas, con un puntero o **hand** que apunta a la página “más vieja” candidata. Ante un page fault se verifica el bit A de la página apuntada por *hand*: si es cero, se la elige; si no, se hace $A = 0$ y se avanza *hand* repitiendo el proceso. Esta implementación se conoce como **algoritmo del reloj** (*clock*).

7.3. Menos recientemente usada (LRU)

LRU (*Least Recently Used*) busca como víctima una página que no ha sido usada en el último lapso de tiempo.

Idea pura (costosa). Mantener una lista ordenada de todas las páginas por tiempo de acceso (de menor a mayor). En cada intervalo de tiempo (*ticks*), las páginas accedidas se mueven al final. Ante un page fault, la víctima es la cabeza. Sin soporte de hardware específico, esta implementación es inviable: cada acceso a memoria debería actualizar la lista.

Implementación práctica: NFU. Una implementación alternativa se conoce como **Not Frequently Used** (NFU):

- A cada página se le asocia un *contador* inicialmente en cero.
- Periódicamente, el SO recorre cada *pte* de cada tabla de páginas y *suma el bit A al contador*.
- Se mantiene una lista ordenada por contador, de menor a mayor.
- Ante un page fault, la víctima es la página a la cabeza (menor contador).

Dos problemas de NFU y sus soluciones.

1. **Recorrer todas las tablas periódicamente es caro.** Se mitiga haciéndolo en momentos de *ocio* del sistema y en forma *incremental*.
2. **El contador sólo crece: nunca olvida.** Páginas que fueron muy usadas en el pasado pero ahora no lo son, quedarían protegidas para siempre. Se resuelve con **aging**: antes de sumarle el bit A , se hace un *shift a derecha* del contador (dividir por dos). Así, los accesos antiguos pesan cada vez menos.

Caso de estudio: Linux. El kernel de Linux usa actualmente una versión de *software LRU* (NFU) de **dos niveles**, manteniendo dos listas: **active_list** (páginas accedidas recientemente) e **inactive_list**. Las páginas inactivas son las candidatas a ser reemplazadas.

7.4. No recientemente usada (NRU)

NRU (*Not Recently Used*) selecciona una página a reemplazar usando exclusivamente el soporte de los bits A (accessed) y D (dirty) del hardware.

El objetivo es identificar qué páginas fueron accedidas en el último período de tiempo. Periódicamente, cada n ticks, el bit A de cada *pte* se pone en cero (*cleared*).

Clasificación en clases. Ante un page fault, las páginas se clasifican en 4 clases:

1. $A = 0$, $D = 0$: no accedida en el último período (mejor candidata).
2. $A = 0$, $D = 1$: escrita en el período anterior; el $A = 0$ es producto del *clearing* periódico.
3. $A = 1$, $D = 0$: leída en el último período.
4. $A = 1$, $D = 1$: accedida en el último período, y además fue escrita en algún momento (peor candidata).

Se elige una víctima en la **menor clase no vacía**. Este algoritmo se conoce también como *software LRU* porque permite implementar una aproximación de LRU en arquitecturas modernas *sin* soporte específico para LRU.

7.5. Working set

Definición

El **working set** de un proceso es el conjunto de páginas a las que el proceso accedió en un *período de tiempo* reciente τ .

Idea intuitiva

Los programas raramente acceden a su memoria al azar: en una iteración o cuando trabajan sobre datos contiguos, referencian *muchas veces* un mismo conjunto de páginas. Mantener ese conjunto en RAM y desalojar el resto es una excelente heurística.

Implementación. Se asocia a cada página un *reloj lógico* (en ticks) que registra el tiempo del último acceso (*tick*).

Ante un page fault se recorre la tabla de páginas inspeccionando el bit A de cada *pte*:

- Si $A = 1$, se actualiza el reloj asociado a la página y se agrega al working set del período corriente.
- Si $A = 0$ se evalúa si puede elegirse como víctima:
 - Si su tiempo de acceso cae dentro del intervalo actual ($tick - last_access_{page} < \tau$), se mantiene en el working set.
 - Si no, se considera *fuera* del working set.

Se elige como víctima una página fuera del working set con menor valor de reloj.

Para evitar recorrer toda la tabla de páginas en cada falta, puede combinarse con una mejora del estilo del *algoritmo del reloj* descrito antes.

El algoritmo de Linux, descrito en § 8, intenta de alguna manera que la lista de páginas *activas* represente el working set actual del sistema.

7.6. Otras consideraciones

Los algoritmos descriptos arriba no consideran ciertos aspectos adicionales que el SO real sí debe tener en cuenta:

Política global vs. local. Hay que decidir si la política de reemplazo es:

- **Global:** no se considera a qué proceso pertenece la página candidata.
- **Local:** sólo se buscan víctimas entre las páginas *del proceso corriente*.

No es lo mismo reemplazar una página del kernel o de una biblioteca compartida, que una página de un proceso de usuario.

Páginas protegidas. Para evitar problemas, ciertas páginas se **protegen** y se marcan para que no puedan ser elegidas como víctimas. Por ejemplo, las páginas del kernel o las que pertenecen a espacios de direcciones de dispositivos mapeados en memoria (MMIO), donde escribir/leer tiene efectos físicos sobre el hardware.

Trashing (sobrepaginación). Uno de los peligros del *page swapping* es justamente este fenómeno.

Importante

La **sobrepaginación** (*trashing*) es un estado en el que el sistema está corriendo *out of memory* y ocurren repetidamente los siguientes eventos:

1. Se hace un swap-out para liberar memoria.
2. La página recién desalojada vuelve a ser referenciada y hay que cargarla de nuevo (swap-in).

Estos pasos se repiten a alta frecuencia; el sistema degrada el rendimiento drásticamente y el progreso de los procesos se hace muy lento.

8. Caso de estudio: GNU/Linux

Linux implementa un algoritmo **LRU**, aunque la lista de páginas no refleja exactamente un orden estricto por tiempo de acceso.

Mantiene dos listas:

- **active_list**: idealmente contiene un *working set* de todos los procesos.
- **inactive_list**: lista de candidatos a ser reemplazados.

Estas listas se actualizan periódicamente. Esto hace que la política sea **global**: cualquier página candidata puede ser elegida, independientemente de a qué proceso pertenece.

9. Cierre

La memoria virtual es la pieza que conecta el paginado del hardware con la realidad de los sistemas modernos, donde la RAM siempre es menos abundante que la memoria que los procesos pretenden usar.

- El **swapping / demand paging** permite que la RAM se comporte como una caché del espacio de direcciones lógicas: el bit *V* y el *disk block number* en la *pte* indican si una página vive en RAM o en disco.
- El **mapeo de archivos** (**mmap**) usa la misma maquinaria para exponer archivos como arreglos de memoria. La carga se difiere hasta el primer acceso, y el bit *dirty* marca qué páginas hay que escribir de vuelta al cerrar.
- La **carga bajo demanda** y el **crecimiento dinámico del stack** reducen drásticamente el costo de **exec()** y eliminan la necesidad de reservar pila por adelantado.
- La **memoria compartida** (**shmget**, **shmat**, **shmdt**) y las bibliotecas compartidas se apoyan en mapear el mismo **ppn** en varias tablas de páginas; un contador de referencias resuelve la liberación segura.
- El **copy on write** es lo que hace que **fork()** sea barato en la práctica: el padre y el hijo comparten la memoria en sólo-lectura hasta que alguien escribe.
- Los **algoritmos de reemplazo** (FIFO, second chance/clock, LRU/NFU con aging, NRU por clases, working set) son aproximaciones distintas al mismo problema imposible: adivinar qué página no se va a usar más. Cada uno equilibra la precisión de la heurística contra el costo de implementarla.
- **Linux** combina varias de estas ideas en un esquema de dos listas (**active/inactive**) con política global, intentando aproximar el working set del sistema.

Idea intuitiva

La memoria virtual no es “un truco” aislado: es una capa que se construye sobre el paginado y vuelve a usar las mismas herramientas (bits V , A , D , *page faults*) para resolver problemas muy distintos: ejecutar procesos grandes, compartir bibliotecas, abaratar `fork`, mapear archivos, hacer crecer el stack. El próximo capítulo (*Entrada/Salida*) se apoya también en muchas de estas ideas, especialmente en el mapeo en memoria de dispositivos.